

EE5332 Project Report- 2D Convolution

TEAM MEMBERS:

K V Veerendra Babu EE25M102

Daniel Mark Isaac NS24Z353

C V Tirumala Rao EE25M067

SLIDING WINDOW + LINE BUFFER Version for 2D convolution

Execution procedure:

Line buffers of size (kernel_size - 1, MAX_IMAGE_WIDTH) are mapped to BRAM using #pragma HLS BIND_STORAGE with ram_2p. For MAX_IMAGE_WIDTH = 256, each row uses 9,216 bits, fitting within a single 18K BRAM. With complete partitioning along dim=1, each row maps to an independent BRAM, totaling (kernel_size - 1) BRAM_18K blocks.

Although ram_2p (true dual-port) allows flexible read/write access, the same II=1 performance was achieved with ram_s2p. To maintain II=1, both window and kernel_in arrays are fully partitioned into registers, enabling parallel access. Image size is provided via AXI-Lite.

Gray scale images are generated using randint(0, 255) and are divided by 255. Kernel ranges are from -1 to 1, generated using uniform distribution or specified explicitly. Expected output image is computed using scipy.ndimage.convolve.

FLOAT:

For all the solutions (unless specified):

Target time = 10ns

Uncertainty = 1.25ns

Trip count: min=66 max=130

Solution1:

set_directive_pipeline -II 1 "conv2d/COL_SLIDE"

Modules & Loops	Issue Type	Slack	Latency (cycles)	Latency (ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
+ conv2d	-	0.24	16952	1.695e+05	-	16953	-	no	2 (~0%)	48 (21%)	6094 (5%)	8108 (15%)	-
o ROW_SLIDE_COL_SLIDE	-	8.75	16949	1.695e+05	51	1	16900	yes	-	-	-	-	-

Solution2:

set_directive_allocation -type operation -limit 1 "conv2d/COL_SLIDE" fmul

set_directive_allocation -type operation -limit 1 "conv2d/COL_SLIDE" fadd

Modules & Loops	Issue Type	Slack	Latency (cycles)	Latency (ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
+ conv2d	-	0.24	152143	1.521e+06	-	152144	-	no	2 (~0%)	8 (3%)	2954 (2%)	2510 (4%)	-
o ROW_SLIDE_COL_SLIDE	II	8.75	152140	1.521e+06	50	9	16900	yes	-	-	-	-	-

Solution3:

set_directive_pipeline -off "conv2d"

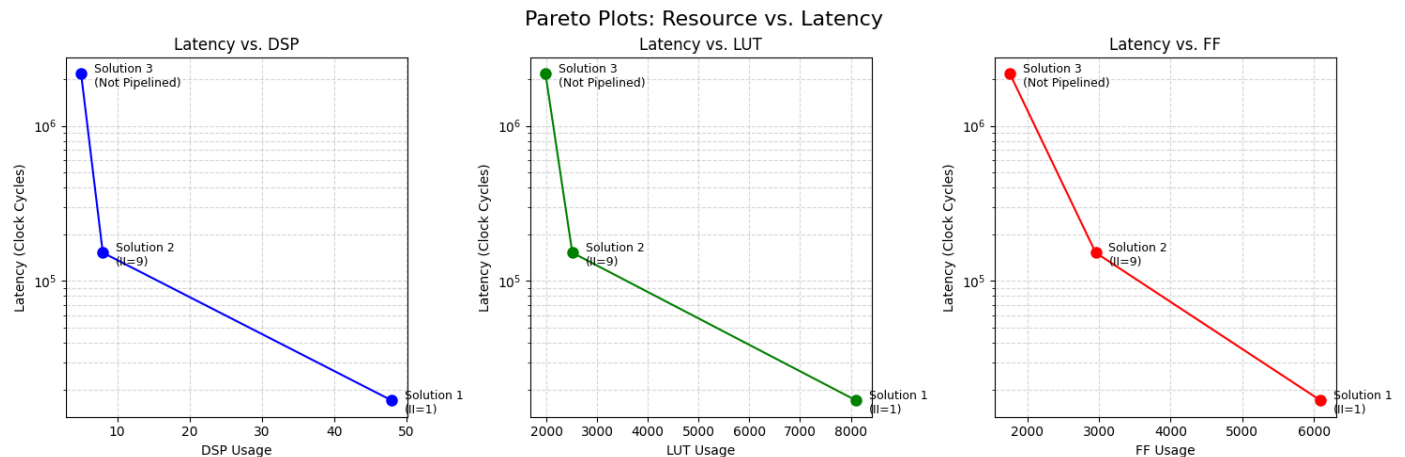
Modules & Loops	Issue Type	Slack	Latency (cycles)	Latency (ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
+ conv2d	-	1.49	2163461	2.163e+07	-	2163462	-	no	2 (~0%)	5 (2%)	1757 (1%)	1982 (3%)	-
o ROW_SLIDE	-	8.75	2163460	2.163e+07	16642	-	130	no	-	-	-	-	-
o COL_SLIDE	-	8.75	16640	1.664e+05	128	-	130	no	-	-	-	-	-
o UPDATE_WINDOW_ROW	-	8.75	12	120.000	4	-	3	no	-	-	-	-	-
o UPDATE_WINDOW_COL	-	8.75	2	20.000	1	-	2	no	-	-	-	-	-
o UPDATE_BUF	-	8.75	6	60.000	3	-	2	no	-	-	-	-	-
o MAC1	-	8.75	105	1.050e+03	35	-	3	no	-	-	-	-	-
o MACj	-	8.75	33	330.000	11	-	3	no	-	-	-	-	-

Solution4:

Target Time = 40ns

set_directive_pipeline -II 1 "conv2d/COL_SLIDE"

Modules & Loops	Issue Type	Slack	Latency (cycles)	Latency (ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
+ conv2d	-	3.79	16921	6.768e+05	-	16922	-	no	2 (~0%)	48 (21%)	5220 (4%)	8063 (15%)	-
o ROW_SLIDE_COL_SLIDE	-	38.75	16919	6.768e+05	21	1	16900	yes	-	-	-	-	-



Trade off:

	BRAM	DSP	FF	LUT
Kernel = 3, float	2	48	6094	8108
Kernel=5, float	4	148	18010	25060

Float on pyng:

For kernel = 3, in float version my max burst size is 16. For all other versions it is 256.

	img: 64, k: 3	img: 128, k:3	img: 64, k: 5	img: 128, k: 5
Scipy time (in us/sample)	0.689	0.545	0.986	1.43
Perf_counter_dma (in us/sample)	0.8	0.4	0.7	0.2
Axi timer dma (in us/sample)	1.325	0.113	0.514	0.08
Max Absolute error	1.97e-7	5.36e-7	1.05e-6	7.59e-7

BRAM usage will be kernel_size – 1, as long as the image size is less than 512. We kept max image size to 256.

FIXED POINT IMPLEMENTATION:

Execution procedure for fixed point:

Fixed point operations are light weight in hardware compared to floating point operations and require lesser number of DSPs. The same sliding window+line buffer architecture is used. When working with fixed point, we need to decide precision.

When kernel size is 3x3, ap_fixed<16,5> is chosen. Because if input pixels are all ones (normalized) and all 9 kernel values are ones (kind of averaging filter without scaling) the MAC output will be 9. To represent 9 in signed notation we need 5 integer bits hence ap_fixed<16,5> is minimum requirement.

When kernel size is 5x5, ap_fixed<16,6> is chosen. Because if input pixels are all ones (normalized) and all 25 kernel values are ones (kind of averaging filter without scaling) the MAC output will be 25. To represent 25 in signed notation we need 6 integer bits hence ap_fixed<16,6> is minimum requirement.

All the other solution pragmas are chosen same as floating point to have a fair comparison

Solution1:

```
set_directive_pipeline -II 1 "conv2d/COL_SLIDE"
```

Modules & Loops	Issue Type	Slack	Latency (cycles)	Latency (ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
+ conv2d	-	0.24	16915	1.692e+05	-	16916	-	no	2 (~0%)	12 (5%)	1520 (1%)	1229 (2%)	-
o ROW_SLIDE_COL_SLIDE	-	8.75	16912	1.691e+05	14	1	16900	yes	-	-	-	-	-

Solution2:

```
set_directive_allocation -type operation -limit 1 "conv2d/COL_SLIDE" mul
```

```
set_directive_allocation -type operation -limit 1 "conv2d/COL_SLIDE" add
```

Modules & Loops	Issue Type	Slack	Latency (cycles)	Latency (ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
+ conv2d	-	0.77	-	-	-	-	-	no	2 (~0%)	1 (~0%)	1151 (1%)	931 (1%)	-
o ROW_SLIDE	-	8.75	-	-	-	-	-	no	-	-	-	-	-
o COL_SLIDE	-	8.75	-	-	11	-	-	no	-	-	-	-	-

Solution3:

```
set_directive_pipeline -off "conv2d"
```

Modules & Loops	Issue Type	Slack	Latency (cycles)	Latency (ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
+ conv2d	-	1.44	1064961	1.065e+07	-	1064962	-	no	2 (~0%)	1 (~0%)	1036 (~0%)	1282 (2%)	-
o ROW_SLIDE	-	8.75	1064960	1.065e+07	8192	-	130	no	-	-	-	-	-
o COL_SLIDE	-	8.75	8190	8.190e+04	63	-	130	no	-	-	-	-	-
o UPDATE_WINDOW_ROW	-	8.75	12	120.000	4	-	3	no	-	-	-	-	-
o UPDATE_WINDOW_COL	-	8.75	2	20.000	1	-	2	no	-	-	-	-	-
o UPDATE_BUF	-	8.75	4	40.000	2	-	2	no	-	-	-	-	-
o MACi	-	8.75	42	420.000	14	-	3	no	-	-	-	-	-
o MACj	-	8.75	12	120.000	4	-	3	no	-	-	-	-	-

Solution4:

Time = 40ns

```
set_directive_pipeline -II 1 "conv2d/COL_SLIDE"
```

Modules & Loops	Issue Type	Slack	Latency (cycles)	Latency (ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
+ conv2d	-	26.69	16914	6.766e+05	-	16915	-	no	2 (~0%)	12 (5%)	1351 (1%)	1238 (2%)	-
o ROW_SLIDE_COL_SLIDE	-	38.75	16912	6.765e+05	14	1	16900	yes	-	-	-	-	-

Fixed on pynq:

Time (in us/sample)	img: 64, k: 3	img: 128, k:3	img: 64, k: 5	img: 128, k: 5
Scipy time	0.616	0.291	0.750	0.406
Perf_counter_dma	0.616	0.291	0.750	0.406
Axi timer dma	0.230	0.056	0.252	0.062
Max abs error	0.002248	0.002355	0.009494	0.009210

Trade off:

	BRAM	DSP	FF	LUT
Kernel=3, fixed	2	12	1520	1229
Kernel=5, fixed	4	28	4171	2393

Design Exploration and Performance Analysis of Separable 2D Convolution on FPGA

1. Introduction

Separable 2D convolution is a key optimization in digital signal processing and computer vision. This report explores the implementation of separable convolution on an FPGA platform, focusing on Pareto optimizations for fixed and floating-point arithmetic, resource scalability with kernel size, and hardware performance on the Pynq-Z2 board.

2. Pareto Design Space Exploration

We explored the design space to balance throughput (Iteration Interval) against resource usage (DSP/BRAM) for both 16-bit fixed-point and 32-bit floating-point designs. Our goal was to reach peak throughput (II=1), which was achieved by unrolling and pipelining the architecture. While Pareto 1 provides the minimum iteration interval, Pareto points 2 and 3 demonstrate how throughput is sacrificed to reduce resource utilization under stricter hardware constraints. The pareto exploration was performed on a 3x3 kernel.

Table 1: Pareto Exploration- Fixed Point

Design Point	DSP	BRAM	II	Iteration Latency (Cycles)
Pareto 1	6	2	1	7
Pareto 2	3	2	2	8
Pareto 3	1	2	6	6

Table 2: Pareto Exploration- Floating Point

Design Point	DSP	BRAM	II	Iteration Latency (Cycles)
Pareto 1	32	2	1	29
Pareto 2	15	2	2	31
Pareto 3	5	2	6	31

3. Resource Tradeoffs with Kernel Size

We evaluated the scalability of the design by increasing the kernel size from 3x3 to 5x5. The increase in kernel size directly impacts the number of parallel multiplications and the memory required for line buffers.

Table 3: Resource Utilization vs. Kernel Size

Type	Kernel Size	DSP	BRAM
Fixed	3x3	6	2
Fixed	5x5	10	4
Float	3x3	32	2
Float	5x5	130	4

Transitioning from a 3x3 to a 5x5 kernel results in a roughly linear increase in resources for fixed-point. However, for floating-point, the DSP usage jumps from 32 to 130, indicating a higher overhead for complex parallel structures.

4. Pynq Board Implementation and Performance

The designs were implemented on a Pynq-Z2 FPGA board to measure real-world execution times. An image size of 64x64 pixels was used for testing. For the fixed-point implementation, data was normalized to a range of [0, 1]. To maintain precision, we utilized `ap_fixed<16, 5>` for the 3x3 kernel and `ap_fixed<16, 6>` for the 5x5 kernel. The additional integer bit in the 5x5 configuration accounts for the larger accumulation sum from the larger kernel window.

Table 4: Performance for 64x64 Image

Config	Axi (ms)	Scipy (ms)	HW Perf	Abs Error
Fixed 3x3	1.428	2.8642	3.81	0.00246
Fixed 5x5	1.5304	3.4242	4.05	0.00633
Float 3x3	1.386	3.254	3.61	0
Float 5x5	1.5432	4.024	3.82	0

Table 5: Performance for 128x128 Image

Config	Axi (ms)	Scipy (ms)	HW Perf	Abs Error
Fixed 3x3	1.6752	5.9767	4.43	0.00237
Fixed 5x5	2.168	7.0248	5.94	0.01206
Float 3x3	1.4472	6.8732	3.91	0
Float 5x5	1.625	7.9073	4.4	0

Performance comparison shows that the FPGA accelerator (measured via Axi timer) is significantly faster than the software-based Scipy implementation. For example, in the Fixed 5x5 case, the FPGA execution time is ~1.53ms compared to 3.42ms for Scipy. Max absolute error remains extremely low, validating the fixed-point precision choices.

5. Conclusion

The exploration demonstrates that separable 2D convolution is highly efficient on FPGA. By optimizing the Initiation Interval and selecting appropriate fixed-point bit-widths, we achieve a balance between resource efficiency and performance. The FPGA implementation consistently outperforms the software baseline while maintaining high numerical accuracy.

Contribution:

K V Veerendra Babu, EE25M102 :

- Test files generation in python
- HLS code for Floating point
- Pareto points for floating point
- IP and bitstream generation for floating point
- Pynq deployment for floating point

Daniel Mark Isaac, NS24Z353 :

- Baseline C implementation
- HLS code for Fixed point
- Pareto points for Fixed point
- IP and bitstream generation for Fixed point
- Pynq deployment for Fixed point

C V Tirumala Rao, EE25M067 :

- Seperable convolution analysis
- HLS code for Seperable convolution
- Pareto points for Seperable convolution
- IP and bitstream generation for Seperable convolution
- Pynq deployment for Seperable convolution